Clustering

Table of contents

Packages 1
What gets clustered 2
How many groups 3
K = 1, and K = 2 by decision 4
Groups and tables 5
Validation 8
Summary 8
Save 8

The HR/LR labels come from one outcome chosen in advance. Clustering asks the data instead: if people are grouped by how they responded, do natural groups appear? This script runs for seven groupings, set by the `grouping` parameter:

grouping	people clustered on
`Muscle_size`	% change over training in fibre area (mixed, type I, type II) and muscle area
`Muscle_strength`	% change over training in leg-press and leg-extension 1RM
`Proteome_T1`, `_T2`, `_T3`	the module eigengenes at that time point
`Proteome_training`, `_acute`	the change in module eigengenes over training or the bout

No step here reads a protein contrast.

Packages

cluster supplies PAM and the gap statistic; fpc the bootstrap stability; mclust the adjusted Rand index. fpc and mclust are called with `::`.

```
library(here)
library(dplyr)
library(tidyr)
library(purrr)
library(tibble)
library(ggplot2)
library(cluster)
source(here("00_Setup", "Shared_functions", "analysis_reporting_helpers.R"))
params$grouping
```

```
[1] "Proteome_T1"
```

What gets clustered

Each grouping becomes one or more people-by-feature matrices. Every feature is z-scored (centred and divided by its standard deviation) so no trait or module dominates by its scale. People missing any feature are left out of that matrix.

```
is_proteome <- startsWith(params$grouping, "Proteome_")
out <- here("02_Groups", params$grouping, "c_data")
dir.create(out, recursive = TRUE, showWarnings = FALSE)
inputs <- c(
  modules = if (is_proteome) "01_Processing/06_Module_Building/c_data/modules.rds",
  phenotype = "00_Setup/Input/phenotype.csv"
)
paths <- map_chr(inputs, here)
stopifnot(file.exists(paths))
phenotype <- readr::read_csv(paths[["phenotype"]], show_col_types = FALSE)
hr_lr_of <- set_names(phenotype$group, phenotype$subject)
```

For the two phenotype groupings the features are the training % changes of the chosen traits.

```
traits <- list(
  Muscle_size = c("pct_fcsa_mixed", "pct_fcsa_type1", "pct_fcsa_type2", "pct_mcsa"),
  Muscle_strength = c("pct_leg_press_1rm", "pct_leg_ext_1rm")
)[[params$grouping]]
change <- phenotype_tables(phenotype)$pairings$training
complete <- change |>
  select(subject, all_of(traits)) |>
  drop_na()
x <- scale(as.matrix(select(complete, -subject)))
rownames(x) <- complete$subject
matrices <- set_names(list(x), params$grouping)
head(round(x, 2), 4)
```

For the proteome the features are the 15 module eigengenes, as levels at each time point and as within-person change over training (T2 − T1) and after the bout (T3 − T2). All five matrices are built; the grouping keeps its own.

```
source(here("00_Setup", "Shared_functions", "proteome_inputs.R"))
modules <- readRDS(paths[["modules"]])
meta <- as_tibble(modules$meta)
values <- modules$eigengenes
matrices <- list(
  T1 = values_at_timepoint(values, meta, "T1"), T2 = values_at_timepoint(values, meta, "T2"),
  T3 = values_at_timepoint(values, meta, "T3"),
  training = within_subject_change(values, meta, "T1", "T2"),
  acute = within_subject_change(values, meta, "T2", "T3")
) |>
  map(\(m) scale(t(m)))
stopifnot(identical(names(matrices), inputs_used))
matrices <- matrices[sub("Proteome_", "", params$grouping)]
map(matrices, dim)
```

```
$T1
[1] 15 15
```

How many groups

PAM (partitioning around medoids) picks K real people as group centres and assigns everyone to the nearest centre. PAM always returns K groups, so three diagnostics judge whether they are real:

- Gap statistic (Tibshirani, Walther and Hastie 2001): compares how tight the K groups are with how tight they would be in 500 reference data sets with no group structure. The one-standard-error rule (`maxSE(method = "Tibs2001SEmax")`) picks the smallest K whose gap is within one standard error of the best. K = 1, no groups, is a possible answer.
- Mean silhouette: how much closer people sit to their own group than to the next, from −1 to 1.
- Bootstrap Jaccard (Hennig 2007, `fpc::clusterboot`, 500 resamples): how often each group comes back when people are resampled. Hennig reads 0.75 or more as stable, 0.6 to 0.75 as doubtful and below 0.6 as not to be trusted.

The adjusted Rand index (ARI) against HR/LR describes agreement with the study's labels; 0 is chance agreement, 1 identical. It decides nothing.

`cluster_people()` takes one z-scored matrix and returns the gap table, the K the gap statistic picks, the PAM fits for K = 2 and 3, their stability, and the diagnostics table. It is applied to each matrix of the grouping.

```
pam1 <- \(x, k) list(cluster = pam(x, k, cluster.only = TRUE))
cluster_people <- function(x, k_max = 3, n_boot = 500) {
  set.seed(1)
  gap <- clusGap(x, FUNcluster = pam1, K.max = k_max, B = n_boot)
  gap_table <- as_tibble(gap$Tab) |> mutate(k = seq_len(n()), .before = 1)
  k_gap <- maxSE(gap_table$gap, gap_table$SE.sim, method = "Tibs2001SEmax")
  solutions <- map(set_names(2:k_max), \(k) pam(x, k))
  stability <- map(set_names(2:k_max), \(k) {
    boot <- fpc::clusterboot(x,
      B = n_boot, clustermethod = fpc::claraCBI, k = k, usepam = TRUE,
      seed = 1, count = FALSE
    )
    tibble(k = k, cluster = seq_along(boot$bootmean), jaccard = boot$bootmean)
  }) |>
    list_rbind()
  diagnostics <- gap_table |>
    left_join(
      imap(solutions, \(fit, k) {
        tibble(
          k = as.integer(k), mean_silhouette = fit$silinfo$avg.width,
          sizes = paste(table(fit$clustering), collapse = "/"),
          hr_lr_ari = mclust::adjustedRandIndex(fit$clustering, hr_lr_of[rownames(x)])
        )
      }) |>
        list_rbind(),
      by = "k"
    ) |>
    left_join(
```

```
      summarise(stability,
        min_jaccard = min(jaccard), jaccard = paste(round(jaccard, 2), collapse = "/"), .by = k
      ),
      by = "k"
    )
  list(
    gap_table = gap_table, k_gap = k_gap, solutions = solutions, stability = stability,
    diagnostics = diagnostics, pcs = prcomp(x)
  )
}
fits <- map(matrices, cluster_people)
map(fits, \(fit) select(fit$diagnostics, k, gap, SE.sim, mean_silhouette, sizes, jaccard))
```

```
$T1
# A tibble: 3 × 6
      k     gap SE.sim mean_silhouette sizes jaccard
  <int>   <dbl>  <dbl>           <dbl> <chr> <chr>
1     1  0.0141 0.0565          NA     <NA>  <NA>
2     2 -0.0117 0.0633           0.196 9/6   0.75/0.66
3     3 -0.0210 0.0673           0.177 5/5/5 0.59/0.68/0.63
```

K = 1, and K = 2 by decision

The gap statistic picks K = 1 here (T1: 1). K = 1 means the data show no natural groups, and that is a result in its own right. K = 2 is then imposed by decision (decisions log, 2026-10-06) so the exploratory comparisons can ask what separates the two halves the data suggest. Every result downstream of an imposed split is exploratory; read each split with its bootstrap Jaccard.

```
imposed_k <- 2L
imposed_reason <- list(
  Muscle_size = paste(
    "Gap statistic selects K = 1; K = 2 imposed by decision (2026-10-06). It reproduces the HR/
LR",
    "split exactly, so this grouping repeats that comparison under a size-cluster label."
  ),
  Muscle_strength = paste(
    "Gap statistic selects K = 1; K = 2 imposed by decision (2026-10-06) because both
clusters",
    "are stable (bootstrap Jaccard >= 0.84) and the split is independent of HR/LR.
Exploratory."
  ),
  Proteome = paste(
    "Gap statistic selects K = 1 at every input, which is reported as the result; K = 2
imposed",
    "by decision (2026-10-06) so the exploratory grouping can compare the two halves the
proteome",
    "suggests. Read each split with its bootstrap Jaccard."
  )
)[[if (is_proteome) "Proteome" else params$grouping]]
```

```
k_used <- map_int(fits, \(fit) coalesce(imposed_k, fit$k_gap))
stopifnot(k_used == 2)
map_int(fits, "k_gap")
```

```
T1
 1
```

Groups and tables

The phenotype groupings name their two groups `HR` and `LR`, higher and lower responder, by the mean of the first trait. The folder name keeps them apart from the study's HR and LR. The proteome groups keep PAM's numbering, `C1` and `C2`; the number carries no meaning. The traits a phenotype grouping was built on stay out of its group-by-trait tests; for `Muscle_size` so does the composite, since these clusters reproduce HR/LR.

```
fit <- fits[[1]]
choice <- tibble(
  family = params$grouping,
  n_subjects = nrow(x), traits = paste(traits, collapse = ", "),
  k_gap = fit$k_gap, k_imposed = imposed_k, imposed_reason, k_used = k_used[[1]],
  grouped = k_used[[1]] > 1
)
assignments <- complete |>
  mutate(
    k2 = fit$solutions[["2"]]$clustering, k3 = fit$solutions[["3"]]$clustering,
    cluster = fit$solutions[[as.character(k_used[[1]])]]$clustering
  ) |>
  right_join(select(change, subject), by = "subject") |>
  arrange(cluster, subject)
label_of <- assignments |>
  filter(!is.na(cluster)) |>
  summarise(lead = mean(.data[[traits[1]]]), .by = cluster) |>
  mutate(label = if_else(lead == max(lead), "HR", "LR")) |>
  select(cluster, label)
group_of <- assignments |>
  inner_join(label_of, by = "cluster") |>
  with(set_names(label, subject))
defining_traits <- if (params$grouping == "Muscle_size") c(traits, "comp_hypertrophy") else
traits
saveRDS(
  list(group_of = group_of, groups = c("HR", "LR"), defining_traits = defining_traits),
  file.path(out, "groups.rds")
)
sheets <- list(
  choice = choice, diagnostics = fit$diagnostics, stability = fit$stability,
  assignments = assignments,
  space = as_tibble(fit$pcs$x[, 1:2], rownames = "subject") |>
    left_join(select(assignments, subject, cluster), by = "subject") |>
    mutate(hr_lr = hr_lr_of[subject], group = unname(group_of[subject]), .after = subject),
  space_variance = tibble(
```

```
    component = c("PC1", "PC2"), variance_explained = summary(fit$pcs)$importance[2, 1:2]
  )
)
table(group = group_of, hr_lr = hr_lr_of[names(group_of)])
```

```
per_input <- imap(fits, \(fit, input) {
  x <- matrices[[input]]
  k <- k_used[[input]]
  cluster <- fit$solutions[[as.character(k)]]$clustering
  list(
    choice = tibble(
      input,
      n_subjects = nrow(x), n_features = ncol(x), k_gap = fit$k_gap, k_imposed = imposed_k,
      imposed_reason, k_used = k, grouped = k > 1
    ),
    diagnostics = mutate(fit$diagnostics, input = input, .before = 1),
    stability = mutate(fit$stability, input = input, .before = 1),
    assignments = tibble(
      input,
      subject = rownames(x), hr_lr = unname(hr_lr_of[rownames(x)]),
      k2 = fit$solutions[["2"]]$clustering, k3 = fit$solutions[["3"]]$clustering, cluster
    ),
    space = tibble(
      input,
      subject = rownames(x), hr_lr = unname(hr_lr_of[rownames(x)]),
      cluster, PC1 = fit$pcs$x[, 1], PC2 = fit$pcs$x[, 2]
    ),
    space_variance = tibble(
      input,
      component = c("PC1", "PC2"), variance_explained = summary(fit$pcs)$importance[2, 1:2]
    )
  )
})
sheets <- map(set_names(names(per_input[[1]])), \(sheet) list_rbind(map(per_input, sheet)))
saveRDS(
  list(
    group_of = with(sheets$assignments, set_names(paste0("C", cluster), subject)),
    groups = c("C1", "C2"), defining_traits = character(0)
  ),
  file.path(out, "groups.rds")
)
count(sheets$assignments, input, cluster, hr_lr)
```

```
# A tibble: 4 × 4
  input cluster hr_lr     n
  <chr>   <int> <chr> <int>
1 T1          1 HR        4
2 T1          1 LR        5
```

```
3 T1          2 HR        3
4 T1          2 LR        3
```

```
imap(fits, \(fit, input) mutate(fit$gap_table, input = input, k_gap = fit$k_gap)) |>
  list_rbind() |>
  ggplot(aes(k, gap)) +
  geom_pointrange(aes(ymin = gap - SE.sim, ymax = gap + SE.sim)) +
  geom_line() +
  geom_vline(aes(xintercept = k_gap), linetype = "dashed") +
  facet_wrap(~input, nrow = 1) +
  scale_x_continuous(breaks = 1:3) +
  labs(x = "K", y = "gap") +
  theme_bw(base_size = 10)
```

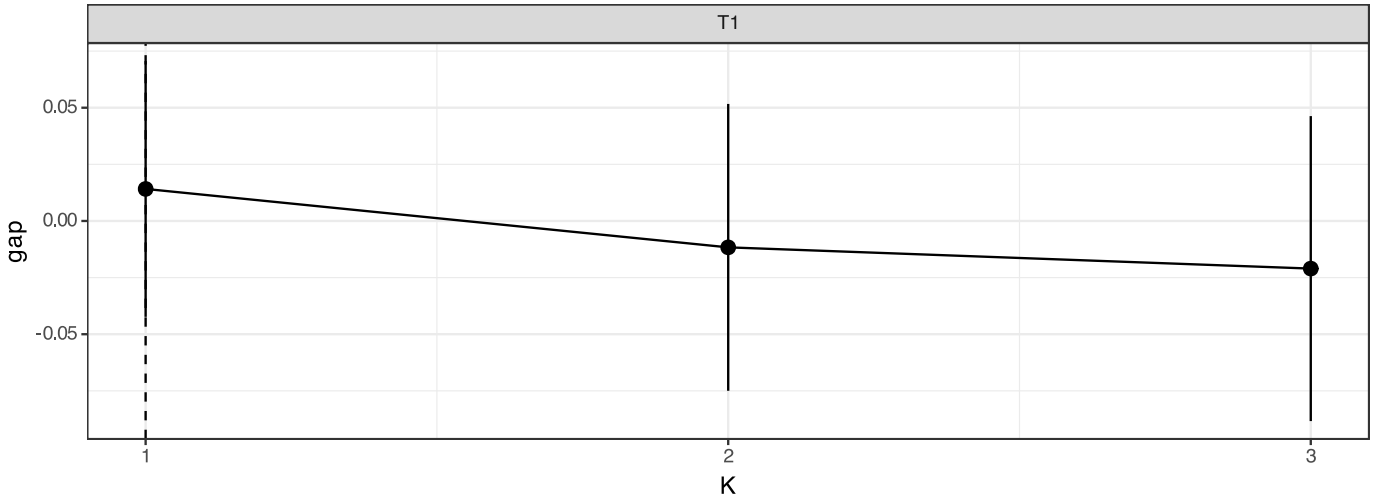


Figure 1: Gap statistic by K, ± one standard error. Dashed: the K the one-SE rule picks.

```
sheets$space |>
  mutate(input = if (is_proteome) input else params$grouping) |>
  ggplot(aes(PC1, PC2, colour = factor(cluster), shape = hr_lr)) +
  geom_point(size = 2) +
  facet_wrap(~input, nrow = 1) +
  labs(colour = "cluster", shape = NULL) +
  theme_bw(base_size = 10)
```

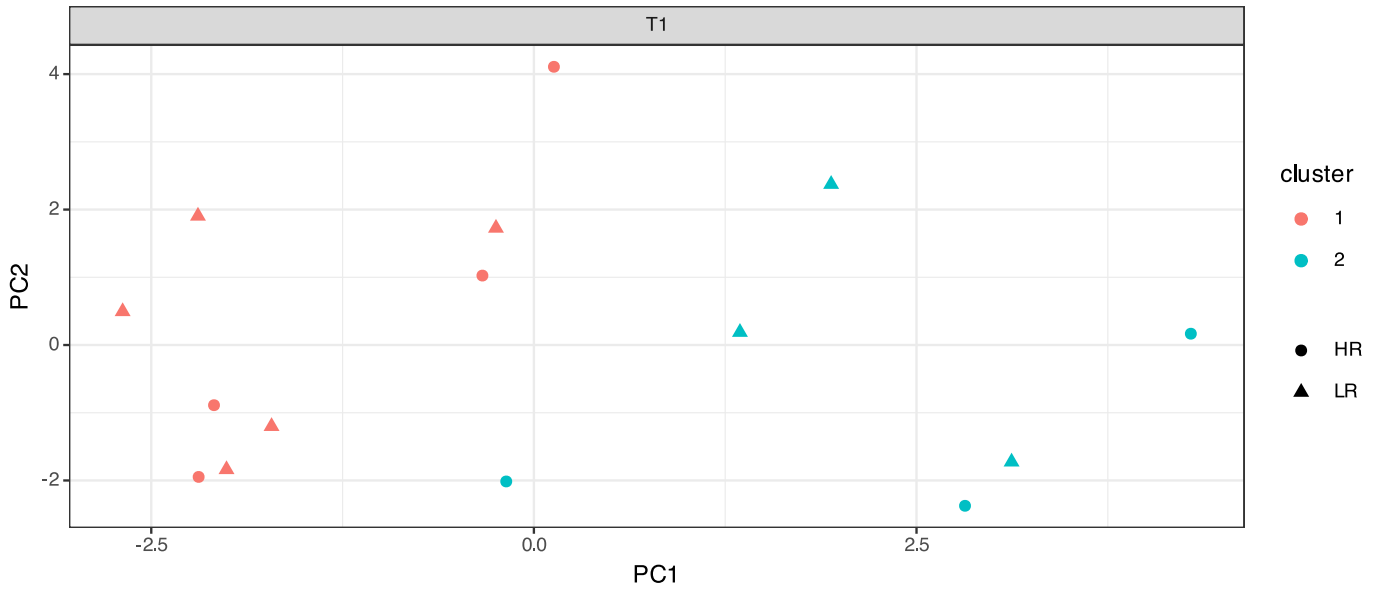


Figure 2: People on the first two principal components of what they were clustered on.

Validation

```
stopifnot(
  all(map_lgl(fits, \(fit) fit$k_gap %in% 1:3)),
  all(sheets$stability$jaccard >= 0 & sheets$stability$jaccard <= 1),
  !anyNA(sheets$space$cluster[!is.na(sheets$space$PC1)])
)
```

Summary

The gap statistic picks K = 1 for Proteome_T1. The imposed K = 2 split has bootstrap Jaccard 0.66 to 0.75 across its groups and matrices; 0 of 2 groups fall below 0.6, Hennig's line for a group not to be trusted.

Save

```
holds <- if (is_proteome) {
  c(
    "Per input: subjects, features, K selected by the gap statistic, any imposed K and
reason.",
    "Per input and K: gap, SE, mean silhouette, sizes, HR/LR adjusted Rand index, Jaccard.",
    "Per input, K and cluster: mean bootstrap Jaccard (fpc::clusterboot, PAM).",
    "Per input and subject: HR/LR (for description), the K = 2 and K = 3 clusters, cluster
used.",
    "Per input and subject: HR/LR, cluster, and PC1 and PC2 of the z-scored eigengenes.",
    "Per input: share of variance on PC1 and PC2 of the z-scored eigengenes."
  )
} else {
  c(
    "K selected by the gap statistic, any imposed K and its reason, and the K used
```

```
downstream.",
    "Per K: gap, SE, mean silhouette, group sizes, HR/LR adjusted Rand index, bootstrap
Jaccard.",
    "Per K and cluster: mean bootstrap Jaccard (fpc::clusterboot, PAM).",
    "Per subject: the traits, the K = 2 and K = 3 clusters, and the cluster used downstream.",
    "Per subject: HR/LR, group, cluster, and PC1 and PC2 of the scaled traits.",
    "Share of variance on PC1 and PC2 of the scaled traits."
  )
}
write_workbook(
  sheets[c("choice", "diagnostics", "stability", "assignments", "space", "space_variance")],
  holds, file.path(out, "clustering.xlsx"), inputs, paths
)
```